Mini Project 1.3: Real-Time Jitter Analyzer Under Simulated System Noise

1. Method

A periodic task (T = 20 ms, 200 jobs per experiment, baseline job body of 4 ms of CPU) runs in a loop that sleeps until the absolute instant r0 + k·T, so it does not drift. **Release jitter** is the actual dispatch instant minus the ideal instant, and **completion jitter** is the deviation of (completion − ideal release) from its mean. Both are reported as standard deviation and maximum absolute value. The deadline is D = 0.8·T = 16 ms after each **ideal** release, so a late start eats into the deadline. Three disturbances can be switched on independently (mini_project_1_3.py, with the loop and load thread in common/rts_common.py):

- **A: background load.** A thread that runs continuous arithmetic for a duty-cycle fraction (0.25 to 1.0) of every 10 ms.
- **B: random extra delay.** A Bernoulli trial before each job starts, with a sleep of configurable length on success. Probability (0.02–0.20 at 5 ms) and magnitude (2–12 ms at probability 0.10) are swept separately.
- **C: execution-time variability.** The job size is drawn from a lognormal distribution (heavy-tailed, median 4 ms) with sigma swept from 0.5 to 1.2. A truncated normal option is also implemented.

The control condition (no disturbance) was run twice, and the loop behaves as intended: release jitter standard deviation 0.03 ms and maximum 0.35 ms, with 100% of deadlines met.

2. Results table

Mechanism	Setting	Rel. mean (ms)	Rel. std (ms)	Rel. max (ms)	Comp. std (ms)	Comp. max (ms)	Hit
baseline (control)	no disturbance	0.15	0.03	0.35	0.59	3.03	100.0%
A: background load	duty 0.25	0.66	0.79	2.57	1.66	4.67	100.0%
A: background load	duty 0.50	2.28	1.74	5.14	2.08	7.15	100.0%
A: background load	duty 0.75	2.85	1.99	5.23	2.81	12.63	97.5%
A: background load	duty 1.00	5.27	0.65	9.15	0.74	4.34	100.0%
B: delay probability	p=0.02, 5 ms	0.23	0.63	5.39	1.21	8.29	100.0%
B: delay probability	p=0.05, 5 ms	0.35	1.01	5.35	1.20	6.09	100.0%
B: delay probability	p=0.10, 5 ms	0.46	1.22	5.41	1.84	8.25	100.0%
B: delay probability	p=0.20, 5 ms	1.30	2.12	5.42	2.57	7.13	100.0%
B: delay magnitude	p=0.10, 2 ms	0.29	0.51	2.31	1.05	4.98	100.0%
B: delay magnitude	p=0.10, 5 ms	0.51	1.32	7.77	1.69	8.48	100.0%
B: delay magnitude	p=0.10, 8 ms	0.70	1.95	8.41	2.29	10.51	99.5%
B: delay magnitude	p=0.10, 12 ms	0.95	2.93	12.36	3.17	14.50	93.5%
baseline (control)	control repeat (sigma=0)	0.14	0.03	0.28	1.07	5.48	100.0%

Mechanism	Setting	Rel. mean (ms)	Rel. std (ms)	Rel. max (ms)	Comp. std (ms)	Comp. max (ms)	Hit
C: exec-time variability	lognormal sigma=0.5	0.14	0.03	0.39	2.32	8.83	100.0%
C: exec-time variability	lognormal sigma=0.8	0.18	0.36	4.72	3.70	19.57	98.5%
C: exec-time variability	lognormal sigma=1.0	0.39	1.97	24.52	5.91	38.35	94.0%
C: exec-time variability	lognormal sigma=1.2	1.34	5.61	49.19	10.32	60.93	86.5%

Table 1: All 18 configurations (Rel. = release jitter, Comp. = completion jitter, Hit = deadline-hit fraction).

3. Sweep plots

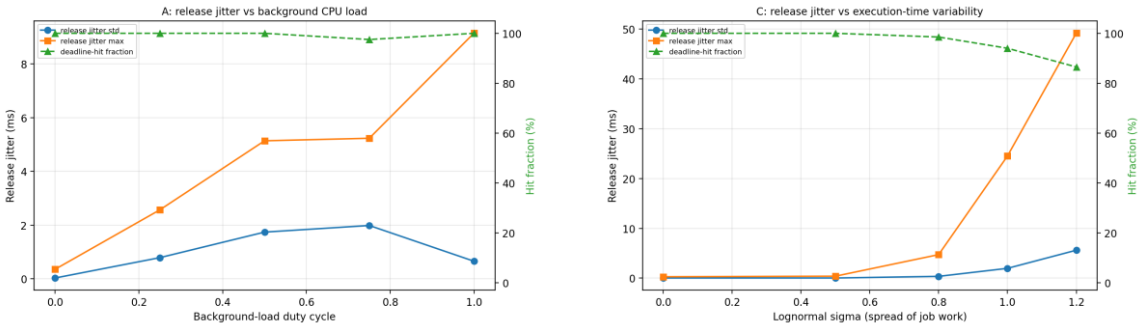


Figure 1 (left): release jitter versus background-load duty cycle. Figure 2 (right): versus execution-time variability. The green dashed line is the hit fraction.

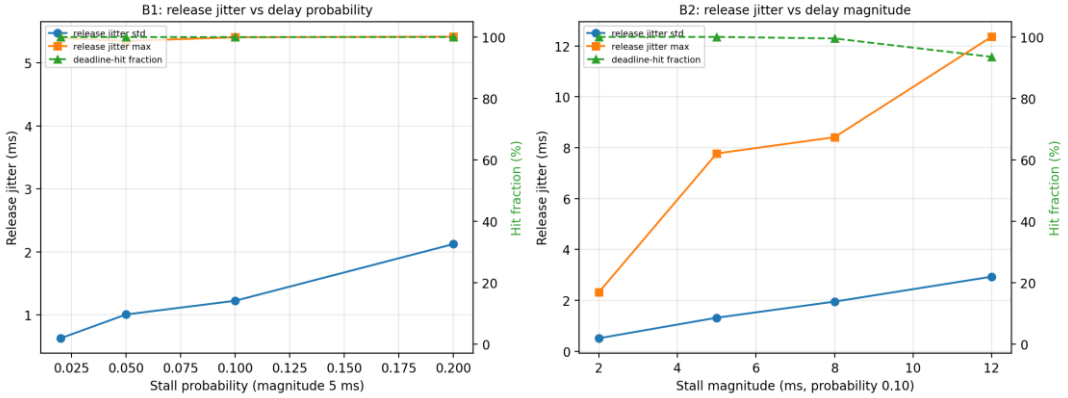


Figure 3: Release jitter versus delay probability (left) and delay magnitude (right).

Background load (Figure 1). Release jitter std rises from 0.03 ms to 1.74 ms at duty 0.50. At duty 1.00 the standard deviation is lower (0.65 ms), but the **mean** delay jumps to 5.27 ms. With a continuously busy competitor every wake-up is delayed by about the same amount, close to CPython's 5 ms thread-switch interval. So the standard deviation understates the damage and the mean and maximum must be read too. **Delay sweeps (Figure 3).** Release jitter grows with both probability (std 0.63 to 2.12 ms) and magnitude (0.51 to 2.93 ms), and its maximum tracks the stall length. Only the 12 ms stall, which nearly exhausts the 16 ms deadline, reduced the hit fraction (93.5%). **Variability (Figure 2).** Release jitter stays near zero until the tail is heavy enough for jobs to overrun the period (sigma ≥ 0.8), while completion jitter

grows immediately: std 2.32 ms at sigma 0.5 against a release std of 0.03 ms. Beyond that, overruns delay following releases and the hit fraction falls to 86.5%.

4. Jitter without missed deadlines

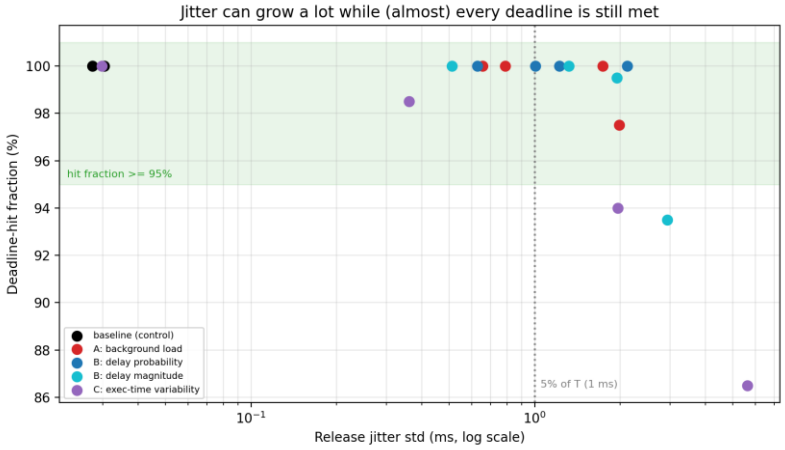


Figure 4: Every configuration as release-jitter std versus hit fraction. The green band is a hit fraction of 95% or more, and the dotted line is 5% of the period (1 ms).

Seven configurations kept at least 95% of deadlines while the release-jitter std was at least 5% of the period (1 ms): background load at duty 0.50 and 0.75, delay probabilities 0.05, 0.10 and 0.20, and delay magnitudes 5 ms and 8 ms. The clearest example is probability 0.20 with 5 ms stalls: the release jitter std is 2.12 ms (10.6% of T, against 0.03 ms in the control) and the hit fraction is still 100.0% (Figure 5). So "meets every deadline" and "has low jitter" are different properties.

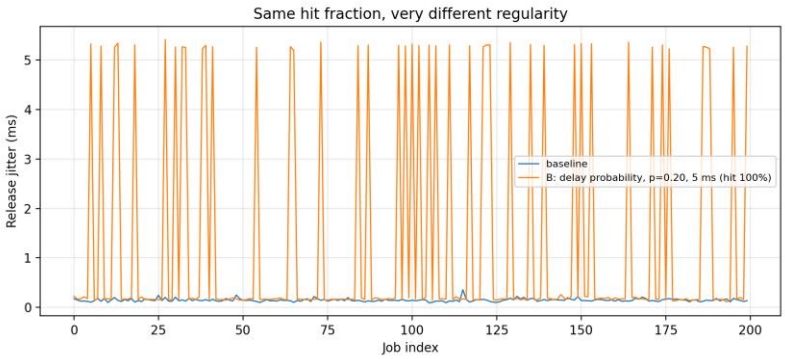


Figure 5: Release jitter over 200 jobs, control versus stall probability 0.20 (same 100% hit fraction).

5. Mapping the disturbances to Section 1.6

Disturbance	Section 1.6 source it approximates	Where the analogy breaks down
A: background load thread	OS jitter and general interference: other tasks, scheduler and shared resources competing with the real-time task.	CPython threads share one interpreter lock, so delays are quantized by the roughly 5 ms switch interval. A real RTOS uses priorities and preemption, and interference would be in caches and buses as well as CPU time. Results also depend on the number of cores (one in my runs).
B: random start delay	Interrupts and DMA stalls that delay a task from starting, for example an interrupt handler or a DMA burst holding the bus.	It is a sleep, so the CPU stays free, whereas a real interrupt or DMA transfer consumes CPU or bus time. The trials are independent, but real

Disturbance	Section 1.6 source it approximates	Where the analogy breaks down
		interrupts are often periodic or bursty, and the delay occurs only before the job starts.
C: execution-time variability	Cache effects and pipeline hazards: the same code takes different times depending on hardware state.	Here the amount of work is drawn from an independent random distribution. Real variability is state-dependent and correlated, with warm versus cold cache and branch history, and does not come from an i.i.d. draw.

Caveats. Each setting was run once (200 jobs), on a single core, in a simulation that cannot give certifiable timing. Run-to-run variation is visible even between the two control runs, so small differences between neighboring settings should not be over-interpreted; the trends (jitter rising with disturbance strength, and hit fraction staying high while jitter rises) are the reliable results.